Introduction to Cryptography: What is Cryptography?

Cyber Fundamentals

What is cryptography?

Cryptography is the processes, practices, and algorithms used to ensure that messages, files, and data are securely exchanged without being viewed by unauthorized third parties. Originally used solely for secure communications as far back as ancient Greece, ciphertext was the initiator of cryptography today.

While in ancient Greece, they used a cipher to convert plaintext messages, in modern times, algorithms and keys are used to encrypt and decrypt data. Computer devices can handle the algorithms for you to make it easier to use strong encryption, but with that also comes the ease of breaking encryption.

Cryptography vs Encryption vs Encoding vs Hashing

When first foraying into cryptography, many words get thrown around – encryption, encoding, hashing – it can feel incredibly overwhelming. These are different, though the lines have been muddied over the years.

Cryptography is the whole field that encompasses multiple technologies, techniques, processes, and more. Cryptography includes encryption, encoding, and hashing.

Encryption is the process of converting data from human-readable to non-human readable through the use of mathematical algorithms and keys. Keys are required to encrypt the data and also to read and reconstruct the encrypted information.

Encoding (or character encoding) is the process of applying a specific code (not necessarily mathematical) of letters, numbers, and symbols to plaintext to turn it into ciphertext. With encoded ciphertext, all the necessary information to reconstruct the message is included.

Hashing is the process of taking a file or some data and representing it with a short hash value. Algorithms often generate these, yet this is not the same as encryption. Encryption is a two-way process, with a key to encrypt and decrypt the data. Hashing has no keys and is often seen as a one-way process. Encryption will protect data, while hashing can confirm that the data hasn't been tampered with.



How does cryptography work?

When looking at how cryptography works, you'll often get an answer which is, technically, how encryption works. As encryption is a large portion of cryptography, they often get lumped together. For this lab and to be consistent with answers you'd get on the internet, it will provide the low down on encryption.

At a high level, encryption works by using an algorithm (of which there are many) and often a key. In some cases, the sender and receiver have the same key (symmetric-key encryption), while others have two different keys for the sender and receiver (asymmetric-key encryption).

How the data is encrypted with these keys depends entirely on the algorithm used. Over time, weaknesses are found with certain algorithms, and others are developed to take their place. Some well-known cryptographic algorithms include:

- Triple DES
- AES
- RSA
- Blowfish
- Twofish

Cryptography and you

Though you may not see it, cryptography is used everywhere, from encrypting your messages on mobile messaging apps to digital certificates and signatures on websites you visit. It keeps your financial information safe and protects your personal and sensitive corporate information.

At the end of the day it:

- Keeps data confidential
- Ensures the integrity of the data
- Authenticates the sender and receiver's identities
- Proves that the sender is the actual sender, known as non-repudiation

Cyber Fundamentals

What is hashing?

Hashing is the process of taking an input of a variable length and mapping it to a fixed-size output value. The output of these functions is often referred to as hashes; for example, a basic hashing algorithm could take a string of text and reduce it down to a single character based on the first letter, so the string we love labs would produce a hash of w. Although this would map data to a fixed-size output, this would be a poor hashing algorithm in practice as there would be lots of **collisions**; for example, the strings where we're going we don't need roads would produce the same hash.

Hashing applications

In computing, there are a number of ways a user may come across hashing algorithms; for example, **hash maps** (or hash tables) are a type of data structure used in programming that use a hashing algorithm to organize the data. Alternatively, hashing algorithms are often used to generate **checksums,** which are used to identify errors when transmitting or storing data.

Cryptographic hash functions

In some instances, you may need to use an algorithm that falls into the category of cryptographic hash functions. As the name suggests, they are hashing algorithms that are deemed suitable for cryptography. This type of hash function may be required when generating digital signatures, securely storing passwords, or fingerprinting files.

For example, hashes are often used to identify potentially malicious files. As it is easy to change the name of a file, attackers can often distribute the same malware using a range of different names. By generating a hash based on the content of a file, security experts can compare these hashes with other samples using services such as VirusTotal

, and identify various strains of malware regardless of the filename used by the attacker.

Informative Alert

What makes a cryptographic hash function?

- It consistently gives the same output for a given input; i.e., it's deterministic.
- It is relatively quick to compute the hash of a given input.
- Different inputs should produce different outputs so there are no collisions.
- Making a small change to the input should produce a significant change to the output; this is sometimes referred to as an ‘avalanche effect’.

- It should be practically impossible to reverse the output to get the input, or produce an input that produces a known output; ideally, the only way to achieve this would be by brute force and comparing the inputs and outputs.

Hashing algorithms

In the section below, we introduce some commonly used hashing algorithms with a high-level overview of where they are used.

MD5

Message Digest 5 (MD5) was designed by Ronald Rivest in 1991. Originally intended to be used as a cryptographic hash function, it has since been found to suffer from several vulnerabilities. It is now commonly used to generate checksums and verify data.

SHA-1

Secure Hash Algorithm 1 (SHA-1) is a cryptographic hash function that takes an input and produces a 40-character hexadecimal number. In recent years many organizations have moved away from using SHA-1 due to security concerns and the risk of collisions.

SHA-256

SHA-256 is one of six hashing functions which forms part of the Secure Hash Algorithm 2 (SHA-2) family. Designed by the National Security Agency (NSA) as a replacement for SHA-1, SHA-2 algorithms are used by a range of protocols including **TLS**, **SSL** and **PGP**.

CRC-16 and CRC-32

Cyclic Redundancy Check (CRC) is an error-detecting algorithm that is often used to identify errors in data that have occurred during storage and transmissions. CRC-32 is used by **Ethernet** and **SATA** compliant devices, whereas CRC-16 is used in a range of applications such as **Bluetooth** and **Modbus** protocol.

Hashing tools

There are several different ways to generate a hash. These have been broken down into **command line tools** and **Graphical User Interface (GUI) tools**. There are many other web-based tools available online that will produce the same results; however, these suffer from the added drawback of sending data to an unknown third party.

Command line

Windows, Linux, and macOS all offer a range of command line tools used to generate hashes.

Since Windows 7, certutil.exe can be used to generate file hashes in Windows environments. The basic syntax for this tool is CertUtil -hashfile <file path> [hash algorithm]. Alternatively, the Get-FileHash cmdlet can be used to generate hashes using **PowerShell**.

In Linux operating systems, a range of commands exist which can be used to generate each type of file hash; for example, md5sum, sha1sum and sha256sum can be used to generate each of the hashes discussed above. On macOS systems, these commands vary slightly. While md5 <file path> is used in a similar fashion to md5sum, SHA-1 and SHA-256 hashes are generated by running the shasum command and specifying the particular algorithm you wish to use; for example shasum -a 1 <file path> for SHA-1 or shasum -a 256 <file path> for SHA-256.

GUI tools

A number of open-source tools are freely available to generate hashes. In this lab we have added **QuickHash** to the desktop which allows users to easily generate a range of hashes on text inputs, individual files, or entire disks.

Informative Alert

Did you know?

When collecting evidence, authorities often generate a hash of any storage devices they collect, such as hard drives. Another hash is then generated at a later date; for example, before a trial so it can be compared against the original. These matching hashes can be used to show that a device has not been tampered with while it has been in custody.

Encoding: What is Encoding?

Cyber Fundamentals

What is encoding?

Encoding is a method of converting data from one format to another to be appropriately (and safely) used by a different system. This could include sending binary data over email or viewing special characters on a webpage. Encoding aims to ensure that information is communicated correctly and is, therefore, readable by a system.

Character encoding, for example, allows computers to "read" text by converting each text character (including all letters, numbers, symbols, and spaces) into binary code.

The purpose of encoding is not to keep information confidential. For this reason, it does not protect or secure data. As a result, the transformation of data from one format to another

uses a publicly available system that can be used to reverse data back to its original form quickly. Unlike encryption, encoding does not require a key; to decode data, you only need to know the algorithm used to encode it.

Some examples of encoding methods include **binary**, **hexadecimal**, and **ASCII.**

Encoding and encryption

Encryption and encoding are terms often used interchangeably, but this is not necessarily correct. While encryption involves encoding data, it's only used when data has been encoded to maintain confidentiality and security. Encoded data, on the other hand, is not confidential.

Encryption aims to transform data so that it's only accessible by the intended recipient(s). An example would be sending someone a private message that they can only read or securely sending a password over email. Encoding focuses on making data readable (and useable), but the goal of encryption is to keep data private and unreadable to anyone other than the intended recipient(s).

To learn more about encryption, take a look at the lab linked below:

Prerequisite labs

A list of prerequisite labs to help complete this lab

Title	Difficulty	Time	Status
Introduction to Cryptography: What is Cryptography?	3 Difficulty level 3 of 9	10 Minutes	Completed

Why encode data?

Encoding is an essential part of our everyday lives. It transforms binary data from computers into a form that's readable by required systems. Encoding formats are vital, as they give computers (and us) protocols to view and process raw binary data comprehensibly. Computers are capable of understanding just one language – binary code. Binary employs a system of two symbols, 0s and 1s, to represent all the fundamental data on a computer. Consequently, all computer information must be converted into a sequence of 1s and 0s, a procedure known as serialization. The serialization process must be done to allow the computer to receive the information and revert the encoded data to its initial state.

When data is saved on a computer, it is transformed into a stream of bytes – binary code. After the computer processes this information, it decodes the data from binary and then displays it on the screen. Therefore, data that has been encoded can only be accessed (and, therefore, opened) by a program that is compatible with that particular encoding method. In the case of audio and video files, this is usually done using a codec, an abbreviation for "**coder-decoder**." A codec is an encoding algorithm designed to compress multimedia data, which is then decoded and decompressed during playback.

Various text editing programs support different text encoding types. However, if a program does not support a text file's encoding, it may display some or all characters as odd symbols instead of the intended text.

Thus, when you open a file, your computer retrieves the necessary bytes from your hard drive and decodes all the information into the proper format, determined by the file's extension. For instance, a .mp3 extension signals to the computer that the file contains audio. Different formats, such as MP3, WAV, and AIFF, can store audio files. Other file formats include JPG (for image files), MP4 (for video files), and TXT (for text files).

Encoding data can also compress files into smaller formats to save disk space. This is useful when storing media files, as encoded media files are typically similar in quality to their original uncompressed version but have much smaller file sizes. An example is an MPEG (.MPG) compressed video file. This format only requires a fraction of the disk space the original digital video (.DV) file demands.

Encoding: Binary

Cyber Fundamentals

Binary

Binary is a numbering system invented by Gottfried Leibniz in 1679 that consists of only two numbers or digits: 0 and 1. This numbering system is the basis of binary code, which is the primary language of computers.

The term binary can also refer to any digital encoding/decoding method with only two potential states. The 0 and 1 values in digital data memory, storage, processing, and communications are commonly referred to as '**low**' and '**high**', or '**off**' and '**on**', respectively.

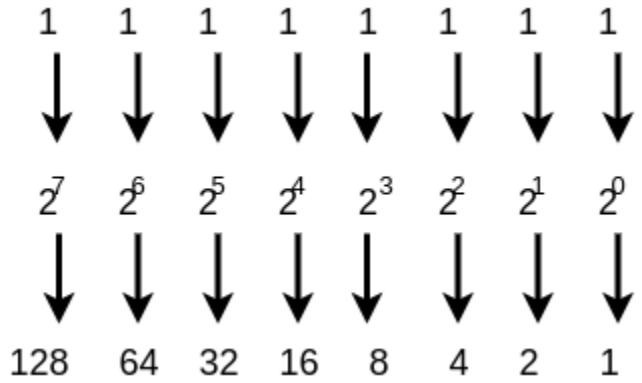
A binary number is made up of eight **bits**, known as a **byte**. A bit, short for **binary digit**, is the smallest unit of data on a computer; each bit has one of two values: 1 or 0. Executable (ready-to-run) applications are identified as binary files with the file extension '.bin.' Programmers often call these executable files **binaries**.

How does it work?

When typed out, binary numbers appear unreadable to the human eye. This is because the weight of the digits grows by powers of two rather than powers of ten. The placement of a binary digit determines its decimal value. For example, if you have an 8-bit binary number, the values will be calculated like so:

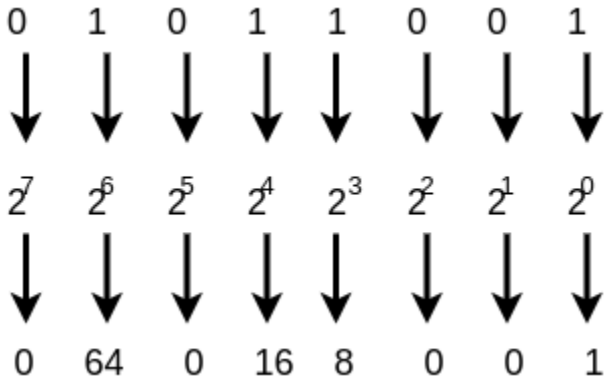
Bit 1: 2^0 = 1
Bit 2: 2^1 = 2
Bit 3: 2^2 = 4
Bit 4: 2^3 = 8
Bit 5: 2^4 = 16
Bit 6: 2^5 = 32
Bit 7: 2^6 = 64
Bit 8: 2^7 = 128

To read binary, you work from right to left. As shown above, the power of the right-most digit is zero, followed (towards the left) by two to the power of one, two, three, and so on. By adding together all these values, you can transform a binary number into a digital number.



You can express any decimal number from 0 to 255 in eight digits of binary by adding the different values when the bit has a value of 1.

For example, the binary number 01011001 represents the decimal number 89 (1+0+0+8+16+0+64+0 = 89).



The table below provides the first 15 decimal numbers in binary form. These can be represented in 4-bits of binary:

Binary	Decimal
0000	0
0001	1
0010	2
0011	3
0100	4
0101	5
0110	6
0111	7

Binary	Decimal
1000	8
1001	9
1010	10
1011	11
1100	12
1101	13
1110	14
1111	15

By adding more bits to the system, you can represent more significant numbers. For example, the decimal number 512 requires nine bits to represent in binary: **111110100**

The transformation of letters into binary is defined by the rules set by UTF-8, in which each character is assigned a group of eight binary digits. For example, the letter A is assigned 01000001. The table below shows the binary representations of A-Z:

Binary	UTF-8	Binary	UTF-8	Binary	UTF-8	Binary	UTF-8
0100 0001	A	0100 1000	H	0100 1111	O	0101 0110	V
0100 0010	B	0100 1001	I	0101 0000	P	0101 0111	W
0100 0011	C	0100 1010	J	0101 0001	Q	0101 1000	X
0100 0100	D	0100 1011	K	0101 0010	R	0101 1001	Y
0100 0101	E	0100 1100	L	0101 0011	S	0101 1010	Z
0100 0110	F	0100 1101	M	0101 0100	T		
0100 0111	G	0100 1110	N	0101 0101	U		

Why is it used and who might use it?

Binary is the primary language for computers and electronic devices, which can only understand and store data in this form.

However, at the most granular level of the data, complex media data such as images and videos are also stored in binary code. An image, for example, is made up of hundreds of thousands of pixels, with each pixel containing an RGB value (color value) stored in binary code.

Music is also stored in binary format, using a technique called pulse code modulation. This technique digitizes continuous sound waves by taking snapshots of their amplitudes every few milliseconds and storing amplitudes of the snapshots in binary. Each second of sound could have a possible 44,000 binary strings. The binary code of an audio file allows a computer to determine the frequency of the vibration in the coils of the speaker, which then projects the sounds you hear.

Note

When transforming **decimals** to and from binary set the operations '**To Base**' or '**From Base**' to **Radix: 2**.

When transforming **letters** to and from binary set the operations '**To Binary**' or '**From Binary**' as **Delimeter: Space** and **Byte Length: 8**.

Encoding: Hexadecimal

Cyber Fundamentals

What is hexadecimal?

Hexadecimal, also called **base-16** or **hex**, is a number system that uses **16** unique symbols to represent a particular value. These symbols are the numbers **0-9** and the letters **A-F** (in capitals), which represent an equivalent binary or decimal number, starting with the least significant digit on the right-hand side.

The unique hexadecimal symbols appear in the following order:

0 1 2 3 4 5 6 7 8 9 A B C D E F

The decimal 12, for example, would be C in hexadecimal.

Computers only understand **binary code** (or **base-2**), which is challenging for humans to read, so programmers created hexadecimal to simplify the binary system. Hexadecimal takes one hexadecimal digit to represent four binary digits. Whereas eight binary digits represent one byte, only two hexadecimal digits are required to do so.

Hex is often used in computing due to its simplicity and the ease of converting it to binary. To learn more about the binary numbering system, visit the lab linked below:

Prerequisite labs

A list of prerequisite labs to help complete this lab

Title	Difficulty	Time	Status
Encoding: Binary 3 Difficulty level 3 of 9		10 Minutes	Completed

How does it work?

Counting in hex is very similar to decimal, except there are six extra non-numerical digits. Firstly, you count from zero to nine and then from A to F. Once you've reached F, you roll that place back to zero and increment the digit to the left by one.

Decimal	Hexadecimal
0	0
1	1
2	2
3	3
4	4
5	5
6	6
7	7
8	8
9	9
10	A
11	B
12	C
13	D
14	E
15	F

Decimal	Hexadecimal
16	10
17	11
18	12
19	13
20	14
21	15
22	16
23	17
24	18
25	19
26	1A
27	1B
28	1C
29	1D
30	1E
31	1F

Converting hex to/from binary

Converting between hex and binary is simple because each digit of a hexadecimal number represents four bits (a bit being an individual binary digit) of a binary value. So a byte (eight bits) can always be represented by two hexadecimal digits. Therefore, hex is a concise way to represent a byte or group of bytes.

Decimal	Hexadecimal	Binary
0	0	0000
1	1	0001
2	2	0010
3	3	0011
4	4	0100
5	5	0101
6	6	0110
7	7	0111

Decimal	Hexadecimal	Binary
8	8	1000
9	9	1001
10	A	1010
11	B	1011
12	C	1100
13	D	1101
14	E	1110
15	F	1111

Converting from binary to hex

To convert binary data to hex, you work with the assumption that four binary digits represent one hex digit.

These are the steps for converting binary to hex:

1. Split the binary value into groups of four digits, starting at the right-hand side of the data.
2. Replace each group of four digits with its matching hex value in the table above.

For example, what if you wanted to know the hexadecimal representation of the binary number **0110110001001101**?

First, starting at the right-hand side of the binary number, you need to sort binary into groups of four:

- 0110
- 1100
- 0100
- 1101

Then, you use the conversion table to convert these groups into a single hex digit:

- 0110 = 6
- 1100 = C
- 0100 = 4
- 1101 = D

Therefore, the binary number **0110110001001101** is converted to **6C4D** in hex.

Converting from hex to binary

To convert from hex to binary, you simply reverse the process:

1. Split the hexadecimal value into single digits
2. For each hex digit, find the matching four-digit binary value in your table, and replace the one hex value with groups of binary digits.

For example, say you need to convert the hex **DECAF** to binary.

First, sort the string into individual digits:

- D
- E

- C
- A
- F

Now convert each hex digit into groups of four binary bits:

- D = 1101
- E = 1110
- C = 1100
- A = 1010
- F = 1111

Therefore, the hex digit **DECAF** is converted to **11011110110010101111** in binary.

How and why is hexadecimal used?

Computer error codes often use hexadecimal numbers. For example, the STOP codes displayed on the famous Windows Blue Screen of Death are always hexadecimal.



Programmers use hexadecimal digits because their values are shorter than decimal equivalents and are more concise than binary. An example of this would be the hexadecimal value F4240. This value is equivalent to 1,000,000 in decimal and 1111 0100 0010 0100 0000 in binary, but the hexadecimal format allows you to store the same information while using less space.

It is also simple to convert between hexadecimal numbers and binary. Hex can, therefore, be used to represent large binary numbers in just a few digits. This makes it easier to read, write, and understand data, which also reduces the possibility of human error when working with it.

Hexadecimal is commonly used as an HTML color code to represent a specific color on the color chart.

For example, the hex value FF0000 is used to define the color red. Breaking this down, the value becomes FF,00,00. This then defines the amount of red, green, and blue colors in this specific shade (RRGGBB), which in this example is 255 red, 0 green, and 0 blue.

Hexadecimal values can be expressed in two digits up to 255, and HTML color codes use three sets of two digits. This means over 16 million (255 x 255 x 255) possible colors can be represented in hexadecimal format, taking up less space than other formats like decimal or binary.



Note

To convert from hex to decimal, use the operation ‘From Base’ with Radix: 16, or from decimal to hex, use ‘To Base’ with Radix: 16.

To convert from binary to hex, use the operation ‘From Binary’ and ‘To Hex', or for hex to binary use ‘From Hex' and ‘To Binary’.

Encoding: ASCII

Cyber Fundamentals

What is ASCII?

Computers operate using numbers and, therefore, need to convert letters (and other characters) to and from numbers so they can be stored and manipulated. An encoding standard known as the **American Standard Code for Information Interchange (ASCII),** debuted in 1963, is used for this purpose. The code represents **128** characters as numbers, with each character assigned a number from **0 to 127**. The standard ASCII character uses seven bits for each character.

ASCII contains the upper- and lowercase letters of the English alphabet, crucial punctuation markers, mathematical symbols, and 33 control codes for data transfer and text formatting.

The following characters are grouped like so:

- **0-32** and **127** – control codes for data transfer and spaces, tabs, and line breaks.
- **48-57** – digits (numbers).
- **65-90** – upper case letters.

- **97-122** – lowercase letters.
- **33-47, 58-64, 91-96, and 123-126** – punctuation marks, mathematical symbols, brackets, and other key characters.

ASCII table

Each letter is assigned a value according to its position within the ASCII table:

ASCII Table

Dec	Hex	Oct	Char	Dec	Hex	Oct	Char	Dec	Hex	Oct	Char	Dec	Hex	Oct	Char
0	0	0		32	20	40	[space]	64	40	100	@	96	60	140	`
1	1	1		33	21	41	!	65	41	101	A	97	61	141	a
2	2	2		34	22	42	"	66	42	102	B	98	62	142	b
3	3	3		35	23	43	#	67	43	103	C	99	63	143	c
4	4	4		36	24	44	$	68	44	104	D	100	64	144	d
5	5	5		37	25	45	%	69	45	105	E	101	65	145	e
6	6	6		38	26	46	&	70	46	106	F	102	66	146	f
7	7	7		39	27	47	'	71	47	107	G	103	67	147	g
8	8	10		40	28	50	(	72	48	110	H	104	68	150	h
9	9	11		41	29	51	)	73	49	111	I	105	69	151	i
10	A	12		42	2A	52	*	74	4A	112	J	106	6A	152	j
11	B	13		43	2B	53	+	75	4B	113	K	107	6B	153	k
12	C	14		44	2C	54	,	76	4C	114	L	108	6C	154	l
13	D	15		45	2D	55	-	77	4D	115	M	109	6D	155	m
14	E	16		46	2E	56	.	78	4E	116	N	110	6E	156	n
15	F	17		47	2F	57	/	79	4F	117	O	111	6F	157	o
16	10	20		48	30	60	0	80	50	120	P	112	70	160	p
17	11	21		49	31	61	1	81	51	121	Q	113	71	161	q
18	12	22		50	32	62	2	82	52	122	R	114	72	162	r
19	13	23		51	33	63	3	83	53	123	S	115	73	163	s
20	14	24		52	34	64	4	84	54	124	T	116	74	164	t
21	15	25		53	35	65	5	85	55	125	U	117	75	165	u
22	16	26		54	36	66	6	86	56	126	V	118	76	166	v
23	17	27		55	37	67	7	87	57	127	W	119	77	167	w
24	18	30		56	38	70	8	88	58	130	X	120	78	170	x
25	19	31		57	39	71	9	89	59	131	Y	121	79	171	y
26	1A	32		58	3A	72	:	90	5A	132	Z	122	7A	172	z
27	1B	33		59	3B	73	;	91	5B	133	[	123	7B	173	{
28	1C	34		60	3C	74	<	92	5C	134	\	124	7C	174	|
29	1D	35		61	3D	75	=	93	5D	135	]	125	7D	175	}
30	1E	36		62	3E	76	>	94	5E	136	^	126	7E	176	~
31	1F	37		63	3F	77	?	95	5F	137	_	127	7F	177	

There is also an extended ASCII table that contains an additional 128 characters, bringing the total to 256, which can be viewed here.

How does it work?

All computer data is stored as numbers, which means a computer must transform any characters (such as those pressed on a keyboard) into numbers to process them. ASCII is the mapping process that assigns each character to a number. For a character to display on the screen, the computer memory must read its ASCII number and "draw" it as a glyph.



How and when is ASCII used?

All ASCII-capable applications (like word processors) can read or store text information to and from computer memory. Without ASCII, modern computing would be more complex and less streamlined.

ASCII code provides a common language across all computer operating systems to establish this link between the screen and hard drive. This means that regardless of the software installed on your computer (Windows, Mac, Linux), you can read documents, as every software recognizes the binary equivalent of the characters on-screen.

Files using ASCII can also serve as a common denominator for all types of data conversions. For example, if a software program cannot convert its data to another format, both programs can still input and output ASCII files. This makes data conversion possible, even when the two software programs are incompatible.

Encoding: Base64

Practical lab

Base64 Encoding

Base64 is a binary-to-text encoding scheme that represents binary data and transforms it into an ASCII string format. Base64 encoding schemes are used when data needs to be stored and transferred over media designed to deal with text.

How does it work?

Base64 breaks binary data into 6-bit groups of three bytes (24 bits) and represents the groups as characters in ASCII. It does this in **two steps**:

Step one is to break down the binary string into 6-bit blocks. Base64 only uses six bits to ensure encoded data is printable and humanly readable. ASCII special characters are not used in this method.

The **64** characters of Base64 encoding (hence the name Base64) are **10 digits, 26 lowercase characters**, **26 uppercase characters**, the **plus sign** (+), and the **forward-slash** (/). The 65th character, known as the **pad**, is the **equals sign** (=). Pad is used when the last segment of binary data doesn't contain the full six bits.

All 64 characters in Base64 encoding can be found in the following table:

Value	Char	Value	Char	Value	Char	Value	Char
0	A	17	R	34	i	51	z
1	B	18	S	35	j	52	0
2	C	19	T	36	k	53	1
3	D	20	U	37	l	54	2
4	E	21	V	38	m	55	3
5	F	22	W	39	n	56	4
6	G	23	X	40	o	57	5
7	H	24	Y	41	p	58	6
8	I	25	Z	42	q	59	7
9	J	26	a	43	r	60	8
10	K	27	b	44	s	61	9
11	L	28	c	45	t	62	+
12	M	29	d	46	u	63	/
13	N	30	e	47	v	pad	=
14	O	31	f	48	w		
15	P	32	g	49	x		
16	Q	33	h	50	y		

For example, say you want to convert the plaintext string **ab@yz**. The binary stream of this string is:

01100001011000100100000001111100101111010

To encode to Base64, you first break the binary stream into groups of six bits. As the binary doesn't equally divide by six, two 0s have been added to the final grouping:

011000 010110 001001 000000 011110 010111 101000

These binary number groupings translate into the decimal numbers **24, 22, 9, 0, 30, 23,** and **40.**

To find out more about how you convert binary numbers to decimals, please visit the lab on binary code below:

Prerequisite labs

A list of prerequisite labs to help complete this lab

Title	Difficulty	Time	Status
Encoding: Binary			
3	10 Minutes	Completed	

Prerequisite labs

A list of prerequisite labs to help complete this lab

Title	Difficulty	Time	Status

Difficulty level 3 of 9

Step two is to convert these decimals using the Base64 encoding table.

The Base64 table shows you that:

- 24 is **Y**
- 22 is **W**
- 9 is **J**
- 0 is **A**
- 30 is **e**
- 23 is **X**
- 40 is **o**

YWJAeXo= (padded with '=' to account for extra bits added).

This two-step process is applied to the entire binary string that is being encoded.

Why do you need it?

Base64 encoding is used to transfer data over a system that only supports ASCII formats. Examples of this are email messages on **Multipurpose Internet Mail Extension (MIME)** and **Extensible Markup Language (XML)** data. Base64 is also the industry-standard format for SSL certificate content. The most common web servers will generate certificate signing requests and accept SSL certificates in Base64 format.

Additionally, Base64 encoding is used when binary data found in media, such as images or video, is transmitted over systems designed to transmit data in an ASCII (plaintext) format.

Base64 encoding is necessary because problems can occur when media, such as those mentioned above, are transmitted in raw binary format to text-based systems.

Because text-based systems like email interpret binary data as a wide range of characters (including command characters), binary data can be misinterpreted by those systems and

lost or corrupted during media transmission. Sending it as plain ASCII text in Base64 encoded format will avoid such transmission problems.



Encoding: Punycode

Cyber Fundamentals

What is Punycode?

Punycode is an encoding method that converts Unicode characters to a limited **ASCII** (American Standard Code for Information Interchange) character set.

This character set, called the **Letter-Digit-Hyphen (LDH)** subset, consists of:

- Lowercase letters: **a-z**
- Digits: **0-9**
- Special character: **hyphen (-)**

Punycode is primarily used to process **Internationalized Domain Names (IDNs)**, which are limited to ASCII characters, as this allows for Unicode characters to be used in a hostname resolution in ASCII format. For example, if a domain name is comprised of Chinese characters, Punycode will encode them into an ASCII format.

How does it work?

Punycode allows domain names to include non-ASCII characters by creating a **bootstring** algorithm, which encodes Unicode characters used within a limited set of ASCII characters. The algorithm interprets any string passed to it and analyzes it for non-ASCII characters. Then, Punycode goes through a series of steps to create an encoded string usable on ASCII systems. Once complete, the algorithm performs some normalization functions on the string to eliminate ambiguities that may exist in the Unicode-encoded text.

Step one: The process of **normalization** is performed on the string to eliminate any Unicode text ambiguity. Normalization involves converting all characters into lowercase (a more 'normal' format), where applicable. The string is then searched for characters that exist within the ASCII character set. Any characters found within the set are ignored; however, any non-standard ASCII characters are removed and a hyphen is added to the end of the string.

Step two: If any non-standard ASCII characters are found, the prefix **xn--** is added to the string. This signifies that the string contains ACE (ASCII Compatible Encoding) and that the hyphen appended should be interpreted using Punycode rather than as part of the string itself.

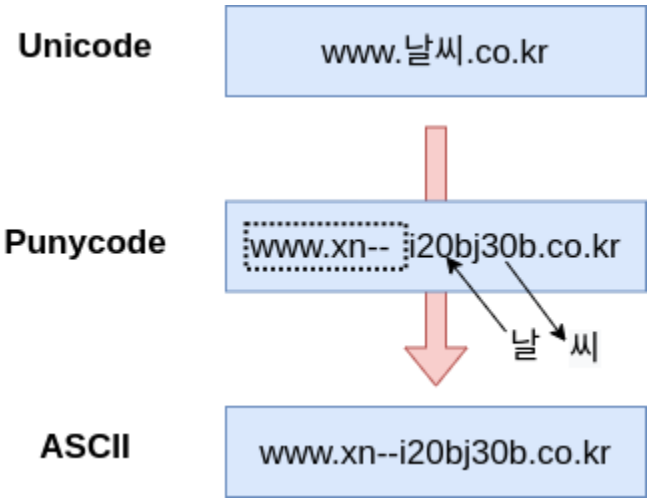
Step three: Punycode analyzes the non-ASCII characters and appends a string of characters to the hyphen. Here, it uses ASCII characters to dictate which characters should be represented and where they should be placed within the string. While doing this, Punycode ensures that the end string does not exceed a 63-character limit.

To see these steps in action, take **adɪdas.com** but with the Greek letter **ɪ** rather than i. Here, the ɪ will be removed, the prefix xn-- will be added to the start of the string to signify the presence of Unicode characters, and a hyphen will be added to the end of the string. Punycode will then add characters to the ending hyphen to represent the Unicode character.

Our final string would be: **xn--addas-rbe.com**

Though the resulting text is not the most straightforward to read, it accurately represents the original string of Unicode characters while using only previously allowed characters for domain names.

Here is an example of the process for a domain name using all Unicode characters:



Punycode allows the encoding of any 8, 16, or 32-bit character. The example above uses the URL www.날씨.co.kr. The characters 날씨 are converted by Punycode to xn--i20bj30b. The newly converted URL can now be represented by www.xn--i20bj30b.co.kr. This means that if you bought the domain name www.날씨.co.kr, you would be actually be buying www.xn--i20bj30b.co.kr.

By default, many web browsers use the xn-- prefix to indicate that the domain is using Punycode to represent some Unicode characters. However, not all browsers show the Punycode prefix, leaving the potential for a **homograph attack.**

Homograph attacks

Though Unicode characters may look the same in a web browser address, their appearance can be deceptive. For instance, many letters in the Roman alphabet look similar to letters in Greek and Cyrillic. This similarity means it can be easy for a malicious actor to replace ASCII characters with Unicode characters in a domain name.

For example, you could swap the English letter 'T' for a Greek Tau: τ – an almost identical symbol to the naked eye. However, the Punycode URL would be xn--5xa rather than just 'T', and if a browser address bar renders in a way that does not indicate that Punycode is present, these character changes can be hard to identify. This ambiguity can lead to a user unintentionally clicking on a harmful link.

The technique of swapping out ASCII characters for Unicode is called a **homograph attack**. The URL will look authentic, and the page content may appear the same, but it will be a different website set up with malicious intentions, such as stealing the victim's sensitive data or infecting their device. Homograph attacks use techniques like phishing, forced downloads, and scams to fulfill these aims.



How and why is it used?

Punycode is valuable for processing internationalized domain names. For example, characters in the Korean character system, Hangul, cannot be adequately encoded using ASCII. Therefore, Punycode takes Unicode encoded strings and converts them into a universally readable and resolvable ASCII string.

Before Punycode, companies and services operating in Korea (or other countries using a character-based language) had to adjust their domain name to fit the ASCII restrictions. For example, 날씨 translates to 'weather' in Korean. Pre-Punycode, a website would have had to change its domain name to an ASCII-acceptable format, such as www.weather.co.kr. Now, a website can use the domain name www.날씨.co.kr intact with Unicode characters. This allows companies and services to keep their brand name identity while operating in markets that do not use the Latin alphabet.

Warning Alert

When using **from Punycode**, ensure the encoding of the output field has been set to **UTF-8**, as the default output (**Raw Bytes**) will not be able to correctly display international characters. You can change this encoding in the toolbar below the output.

Encoding: Unicode

Cyber Fundamentals

What is Unicode?

Unicode is a universal character encoding system that assigns a code to the characters and symbols of most languages. It's the only encoding standard that ensures you can retrieve or combine data using an unlimited combination of languages.

Unicode is used in all major operating systems, search engines, browsers, laptops, and smartphones, as well as on the Internet and the World Wide Web (URLs, HTML, XML, CSS, JSON, etc). As of version 13.1, Unicode now encompasses 144,076 characters used in almost every language in the world, alongside emojis.

To find the Unicode value for emojis, visit the official Unicode website here.

The emergence of Unicode and the availability of tools supporting it are among the most significant recent global software technology trends.

How does it work?

Unicode defines codes (unique numbers) for all characters used in most languages, including diacritics, punctuation marks, mathematical symbols, technical symbols, arrows, and dingbats. Overall, Unicode has codes for over 100,000 characters from world alphabets, ideograph sets, and symbol collections, including the classical and historical texts of numerous written languages.

Unicode can represent characters in different encoding forms, such as UTF-8 and UTF-16.

UTF-8 is a variable-length encoding scheme in which each written symbol is represented by a one to four-byte code, whereas UTF-16 is a fixed-width encoding scheme in which a two-byte code represents each written symbol.



ASCII and Unicode

Unicode and **ASCII** are the most popular character coding standards and are frequently compared as a result. However, they have some essential differences.

Unicode is a universal character encoding method used for the consistent encoding, representation, and handling of text expressed in any spoken language. In comparison, ASCII (American Standard Code for Information Interchange) uses 128 English characters (symbols, letters, digits) as a number to represent text in computers.

A significant difference between these coding methods is size. Unicode represents characters from a more substantial character table than ASCII, which has 128 characters (or 256 with extended ASCII). ASCII represents lowercase letters (a-z), uppercase letters (A-Z), digits (0–9), and symbols such as punctuation marks. In contrast, Unicode represents English, Arabic, Greek, mathematical symbols, historical scripts, emojis, and so on, covering a far greater range of characters than ASCII.

Additionally, the first 128 Unicode characters point to ASCII characters, meaning ASCII can be viewed as a subset of Unicode; this isn't true vice versa.

Unicode also allows characters to be up to 32 bits wide, whereas ASCII uses seven bits (or eight using the 256 version) to represent a character. For Unicode, that’s over four billion unique values to represent all languages accurately. Because Unicode allows for larger bit sizes, it may take up more storage space than ASCII files.

How and why is it used?

The Unicode encoding system provides a consistent method of encoding multilingual plaintext, making it easier to exchange text files internationally. Supporting data with multiple languages such as French, Japanese, and Hebrew, Unicode allows you to combine different language scripts on a single file. Before Unicode, a computer operating system could only process and display the written symbols on its code page, which was tied to one language script. For example, if a computer could process German, it could not process Hebrew or Japanese.

There is a growing trend for new computer technologies to adopt Unicode. Currently, industry leaders such as **Microsoft**, **Apple**, **HP**, **IBM**, and **Oracle** have adopted it. Unicode is also a preferred text encoding method in web browsers **Google Chrome** and **Firefox**, and is used in Java technologies HTML, XML, and Windows, and Office.

Visit the Unicode website here

to learn more.
Warning Alert

When using **Unescape Unicode Characters**, ensure the encoding of the output field has been set to **UTF-8,** as the default output (**Raw Bytes**) will not be able to correctly display international characters. You can change this encoding in the toolbar below the output.

Encryption Tools: CyberChef

Cyber Fundamentals

What is CyberChef?

CyberChef is a simple, intuitive web app developed by GCHQ for analyzing and decoding data, without the need for complex tools or programming languages. It can carry out all manner of cyber operations within a web browser. These include simple tasks like XOR or Base64 and more complex encryption like AES, DES, and Blowfish, alongside binary and hex dumps creation, data compression and decompression, hash and checksum calculations, IPv6 and X.509 parsing, the changing of character encodings, and more.

CyberChef functions can be categorized to:

- Encode/encrypt
- Decode/decrypt
- Format data
- Parse data
- Compress data
- Extract data
- Perform arithmetic functions against data
- Defang data

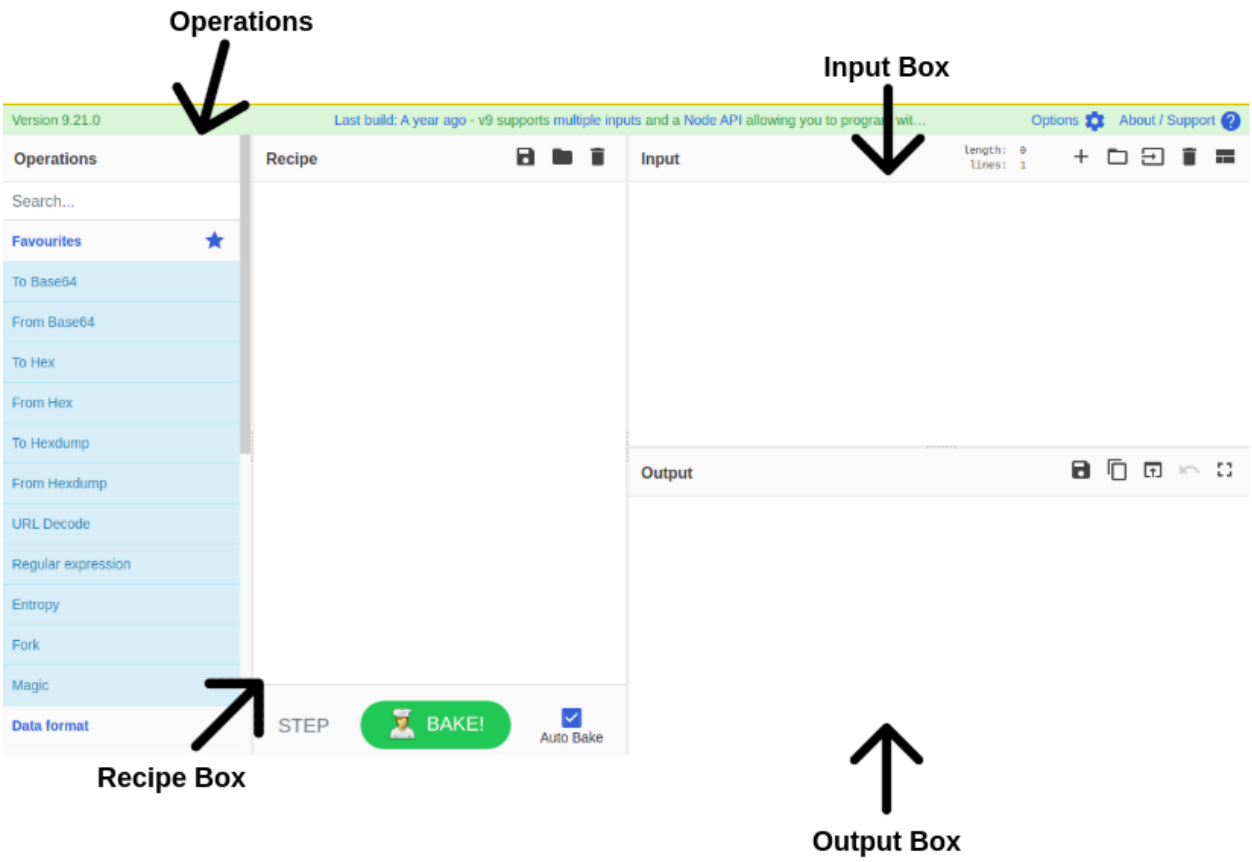
CyberChef can perform many other functions too.

How does it work?

There are four main areas in CyberChef.

1. The **input** box is in the top right. Here you can type, paste, or drag the file or text you want to operate on.
2. The **output** box is in the bottom right. This is where the outcome of your processing is displayed.
3. The **operations** list is on the far left. Here you can find all of CyberChef's operations displayed in categorized lists. These can also be found by using the search bar.
4. The **recipe** box is in the center. Here you can drag the operations you want to use and specify arguments and options.

Operations come in varying levels of complexity and you can use as many as you wish.



To use CyberChef, open the page online or locally (after you've downloaded it onto your system). From there, select the desired **operation** on the left and enter your **input** on the right. You can use the built-in search bar to find certain operations quickly or add items to the Favorites list for easy access.

Operations are sorted into groups with subcategories. For example, the Date and Time group lists five operations: Parse Date Time, Translate Date Time Format, From UNIX Timestamp, To UNIX Timestamp, and Extract Dates.

A double-click will load the selected operation, some of which are customizable. If you select 'From UNIX Timestamp', for instance, you can switch the input unit format from seconds to milliseconds, microseconds, or nanoseconds. From there, you can enter your input manually or by pasting it.

Once the **input** has been entered and the operations selected, the **output** is displayed immediately; no more options need to be selected. You can save the output to a file, copy it to the clipboard, or switch input and output fields.

How and why is it used?

CyberChef encourages both technical and non-technical users to explore data formats, encryption, and compression. Its user-friendly interface means anyone can utilize these features with simple drag and drop operations.

The platform is able to demonstrate a wide range of programming concepts. This includes not only functions, order of operations, and data types, but also the step-by-step visualization of data manipulation.

CyberChef can be downloaded from GitHub to run on your local machine, or it can be used inside a browser here

. **Note**

Hashing functions are very sensitive to changes in the text. Additional lines, for example, will drastically change the output of your result, so take care to copy the message verbatim when you encode it. If you want to learn more about hashing, check out our 'Introduction to Hashing' lab via the link below.

Encoding: Demonstrate Your Skills

Cyber Fundamentals



Introduction

Welcome to the final lab of the **Encoding** collection! In this lab, you have the opportunity to apply and expand on the knowledge you've gained throughout the collection.

By completing this lab, you'll demonstrate your ability to apply various encoding methods to investigate a domain name suspected of containing a homograph attack and successfully decode the message hidden inside. Dive in and put your skills to the test!

Collection recap

In this collection, you explored various encoding methods to convert data into different formats for storage and transfer.

You began by understanding what encoding is and why it's essential in computing and data communication. You then delved into specific encoding types, including binary and hexadecimal, that are foundational for representing data in computing systems.

The collection covered ASCII and Unicode to show how text is represented and manipulated across different platforms and languages. At the same time, you were shown how Base64 encoding was introduced to transmit data over media and was designed to handle textual data efficiently.

Finally, you learned about Punycode, a crucial encoding method for representing domain names with non-ASCII characters. It plays a key role in internationalizing domain names and their use in homograph attacks.

Each lab built your proficiency in recognizing and applying these encoding techniques, which is vital for anyone involved in cybersecurity, data transmission, and software development.

Informative Alert

When entering the Punycode URL into the Firefox address bar, the browser will automatically attempt to redirect you to the HTTPS version of the site. To access the correct page, you'll need to manually adjust the URL in the address bar to use HTTP instead.